

Caruca v2

LLM-based specification mining for opaque shell commands

Naive LLM

Addendum to Caruca · what an undirected agent does with the syntax step, and a command beyond the core utilities · draft 0.2, 2026-10-20

SOURCE	`ai_docs/analysis/white_paper_addendum_naive_llm.md`
GENERATED	October 20, 2026 at 4:21 PM
PAGES	12

Table of contents

Naive LLM: addendum to Caruca

The naive-LLM experiment

1. What an undirected agent does with the syntax step

Summary

1.1 How Caruca's syntax step works today

1.2 The runs

1.3 Case study A: ask the command, not only its manual

1.4 Case study B: grep's -e/-f form

1.5 What this suggests for Caruca

1.6 What this section does not claim

Evidence

2. A command beyond the core utilities: binwalk

Summary

2.1 The run

2.2 Where the syntax came from

2.3 What binwalk needs from the environment

2.4 Two grammar details with consequences for the pipeline

2.5 What this suggests for Caruca

2.6 What this section does not claim

Evidence

Naive LLM: addendum to *Caruca*

To: the Caruca co-authors **From:** Tiran Dagan (Stevens) **Re:** arXiv:2510.14279, *Caruca: Effective and Efficient Specification Mining for Opaque Software Components* **Date:** 2026-10-20 · **Version 0.2, draft** (0.2 adds section 2, a command beyond the core utilities) **Status:** draft for discussion, not yet reviewed by Prof. Greenberg or Prof. Eiers.

Every proposal drawn from the naive-LLM experiment is collected under this heading. The experiment differs from this project's earlier work, which rebuilt Caruca's pipeline stage by stage with prompted models, so nothing here relies on those earlier findings: each claim is backed by the paper, Caruca's code and files, or the naive-LLM runs' own logs.

The naive-LLM experiment

Caruca's syntax step (paper section 3) gives a language model a command's documentation and asks for the command's argument syntax; everything after that step is hand-written procedure. To see what a model does with the problem when it is *not* given Caruca's procedure, we gave a general-purpose coding agent (Claude Code, in a throwaway virtual machine) only a statement of the goal: describe the command's argument syntax in Caruca's format, plus PaSh's annotation files, for downstream tools. It was given the two formats, four finished examples, the path to the binary and the path to its manual page, and told to test the binary against what it produces. Nothing about how Caruca works, no method, no isolation advice, and no web access.

---

1. What an undirected agent does with the syntax step

Two things the agent did that Caruca's design does not do, each as a case study with the session logs as evidence. Both point at small extensions of Caruca's existing architecture rather than at anything Caruca gets wrong: in each case Caruca already holds the material the agent used, and the agent shows a way to put it to work.

Summary

- **Case study A: ask the command, not only its manual.** Caruca's model reads a bundled copy of the manual page and is told not to add anything the manual does not mention. The agent read the manual and then interrogated the binary. Across two commands it established six facts about the argument syntax that Caruca's curated reference files do not record. Two are stated in the manual itself; the other four only the binary reveals, one of them through its own error message.
- **Case study B: grep's `-e` / `-f` form.** `grep` accepts its pattern either as the first operand or through `-e` / `-f`, in which case every operand is a file. All three of Caruca's `grep` grammars hold only the first form, and because the reference shares the gap, the accuracy check reports a match. The agent, without the web, wrote both forms into its grammar, bringing it into line with PaSh's own hand-written `grep` annotation and with what the binary does.
- **What it suggests:** check the syntax against the binary, using the sandbox Caruca already has; let a command's own error messages list its accepted values; and score syntax against the binary as well as against the reference.

1.1 How Caruca's syntax step works today

From the paper (section 3.2) and the code at commit `d80324073` (`caruca/src/caruca/llm.py`):

- The model (GPT-4o) receives one document per command: a bundled text copy of the manual page, `doc_sources/man/<command>.txt`. The paper describes the input as "`man` or `--help` pages"; the repository also

bundles 109 `--help` files (`doc_sources/helpfiles/`), but no code reads them.
- The model never runs the binary. The binary is first run later, during tracing, and nothing from tracing flows back into the syntax.
- The prompt instructs: “Dont add extra choices that are not mentioned in the manpage.”
- The generated syntax is imported and type-checked, with up to three attempts.
- Accuracy (paper section 7.2) is agreement with curated reference files, `syntax_specs/<command>.py` , built from manual pages (the paper: two graduate students, 80 person-hours; the repository: 91 commits by four authors, with messages such as “Fix syntax_spec ground truths”).

This is a sound design for its purpose: the syntax is fixed, reviewable and reproducible, and anchored to the documented interface. The case studies ask what is left out when the documented interface and the binary’s actual interface differ.

1.2 The runs

Three runs of the agent can be re-read in full from their logs (two earlier runs’ records no longer exist). All used the same first-version prompt, in the same virtual-machine image, with permissions bypassed so the agent could act freely.

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
3	`grep`	Claude Haiku 5.5	open	739 s · $0.96	context only: it found this project’s public notes on `grep` through a GitHub search
4	`grep`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	322 s · $2.54	yes
5	`rm`	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	60 s · $0.80	yes

Both commands were among the four examples, so the agent had a finished answer in front of it: for the syntax, that answer is Caruca’s own reference file, byte for byte. The case studies therefore report where the agent *departed* from the reference, and why.

1.3 Case study A: ask the command, not only its manual

In both runs the agent read the manual, ran `--help` , and then ran the binary on small inputs before writing anything: 72 runs for `grep` , 27 for `rm` . Its probes were aimed at the places a syntax description is hardest to get right: options with optional values, invalid values, negative numbers, abbreviated option names, grouped flags, `--` . In `rm` ’s case it passed a deliberately invalid value (`--interactive=bogus`), and `rm` answered with the complete list of values it accepts.

Six facts about the two commands’ argument syntax, and where each one is recorded:

FACT (GNU COREUTILS 9.4, GNU GREP 3.11)	MANUAL PAGE	CARUCA REFERENCE	BINARY	AGENT'S SYNTAX FILE
`rm --interactive` also accepts `no`, `none`, `yes`	no (lists `never`, `once`, `always`)	no (3 values)	yes, listed in its own error message	6 values
`rm` may be given no file at all (`rm -f` exits 0)	yes: `rm [OPTION]... [FILE]...`	no: at least one file	yes (without `-f` it refuses to run)	zero or more files
`grep -y` means `-i`	no	absent	yes	present
`grep -u` / `--unix-byte-offsets` exist	no	absent	yes, with an "obsolete" warning	present
`grep --color` takes its value only after `=`	yes: `--color[=WHEN]`	value as a separate word	yes: `--color never` searches for "never"	value after `=`
`grep -A`, `-B`, `-C` reject negative numbers (while `-m -1` is legal)	not stated	no lower bound on any of the four	yes	`min=0` on the three, none on `-m`

The manual pages are those bundled in Caruca's repository (`grep` 3.11, the same version as in the virtual machine; `rm` from coreutils 8.32, whose option list matches 9.4's). "Binary" is the agent's own probe, verified in its session log. Caruca's model output for both commands (`outputs/llm-dsl-generation/`) records none of the six either.

What the table shows.

- **Two of the six were in the manual all along** (`rm`'s optional file list and `--color[=WHEN]`). The reference files do not record them, and neither does the model's output from the same manual. Running the binary turned each into a checked fact rather than a reading of documentation notation.
- **Four are in the binary only** (`rm`'s extra values, `-y`, `-u`, the context options' refusal of negative numbers). For the first three, Caruca's syntax step cannot find them by design: its input is the manual, and its prompt forbids values the manual does not list. The agent found `rm`'s extra values in a single invalid call, by letting the command describe its own interface.
- **The yardstick question.** Scored against Caruca's references, the agent's departures count as errors; scored against the binary, they are corrections. The references measure faithfulness to the documentation; the tools that consume Caruca's output run against the binary. Both are legitimate targets, and the paper currently reports only the first.

The agent did not get everything right: in PaSh's parser table it kept the example's separate-word `--color WHEN` and `--interactive WHEN` forms although its own probes showed them wrong for this binary, and it said so in its report. The point of the case study is the method, not a score.

1.4 Case study B: `grep`'s `-e` / `-f` form

`grep`'s manual lists three forms in its synopsis (Caruca's bundled copy, `doc_sources/man/grep.txt`, lines 8–10):

```
grep [OPTION...] PATTERNS [FILE...]
grep [OPTION...] -e PATTERNS ... [FILE...]
grep [OPTION...] -f PATTERN_FILE ... [FILE...]
```

In the second and third forms there is no pattern operand: every operand is a file. Loaded through Caruca's own syntax module, each of its three `grep` grammars has exactly one form, a required pattern operand followed by zero or more files: the model's output (`outputs/llm-dsl-generation/grep.py`), the curated reference (`syntax_specs/grep.py`), and the subset used for PaSh (`pash_syntax_specs/grep.py`). The first two list `-e` and `-f` as options while still requiring the

pattern operand; the third lists neither. Run at one flag at a time, Caruca's generator produces 122 `grep` command lines, and the one using `-e` is `grep -e a a`, in which `grep` reads the second `a` as a file name, not a pattern. Because the reference has the same single form as the model's output, the syntax comparison of paper section 7.2 reports a match: the gap is invisible to the evaluation.

What the agent did (run 4, no web). It wrote two syntaxes sharing one option list: the pattern as the first operand, and `-e` / `-f` with every operand a file. The log shows where this came from, and it was not the synopsis, which the agent never displayed (it printed only the manual's option lines and `--help`, whose usage line shows one form). Two sources in front of it carried the fact:

1. **PaSh's hand-written `grep` annotation**, given to the agent as part of the example, already handles it: if `-e` or `-f` is present, every operand is an input file.
2. **The binary**: its probe `grep -e beta t.txt t.txt` searched both operands as files.

So the agent did not discover the form from nothing. It noticed that two descriptions of the same command disagree (the grammar has one form, the behaviour annotation two), checked which one the binary agrees with, and made the grammar match. In Caruca's architecture the grammar and the annotations are produced in different stages and never compared with each other, so this kind of disagreement has no place to surface.

(Run 3, with web access, also wrote several forms, but only after reading this project's own notes on the gap through a GitHub search, so it is not used as evidence.)

1.5 What this suggests for Caruca

Each suggestion reuses something Caruca already has.

1. **Check the syntax against the binary.** Caruca already runs the real binary in an isolated sandbox during tracing. A short syntax check before tracing (each spelling once, each value form, each documented synopsis form, one invalid value per option) would catch documentation-notation misreads such as `--color[=WHEN]` and missing forms such as `grep -e`. In run 4 the agent's equivalent check was 154 runs of the binary, a few seconds.
2. **Let the command list its own values.** Where a command answers an invalid value with the list of valid ones, as GNU tools commonly do, that list is better evidence than the manual's. This needs the prompt's "no choices not in the manpage" rule relaxed to "no choices not confirmed by the manpage or the binary".
3. **Report syntax accuracy against two yardsticks.** Agreement with the reference (as now) and agreement with the binary, with the disagreements between them listed as a category of their own. That turns the 80 person-hours of reference annotation into a measurement of the documentation as well as of the model.
4. **Cross-check the grammar against the consumer's annotation where one exists.** PaSh's hand-written annotations encode facts about argument shape (as for `grep -e`) that a grammar can be checked against mechanically.

1.6 What this section does not claim

- **No rate.** Three analysable runs, two commands, one prompt version; each combination of model and command ran once. These are case studies, not frequencies.
- **No claim the agent is more accurate than Caruca overall.** Neither run's output has been scored, the agent had the reference in front of it, and it made its own mistakes (end of section 1.3).
- **No claim the facts generalise beyond GNU tools.** Case study A relies on informative error messages; an undocumented binary may give none.
- **No claim about the rest of Caruca's pipeline.** The agent was weakest exactly where Caruca is strongest: it isolated nothing beyond a scratch folder, even for `rm`, and in neither run did it check the files it had written against the binary. That contrast is a candidate for a later section under this heading, once repeated runs show whether it is a pattern.

Evidence

CLAIM	SOURCE
Caruca's syntax input, prompt wording, retries	v1 `caruca/src/caruca/llm.py` (lines 28–50, 103–131, 155–159), commit `d80324073`
109 bundled `--help` files, unused by the code	v1 `caruca/src/caruca/doc_sources/helpfiles/`; no `.py` or `.sh` file references them
Reference files are curated	v1 `git log -- caruca/src/caruca/syntax_specs/`: 91 commits, 4 authors
The examples are Caruca's reference files	`naive_llm_caruca/samples/{grep,rm}/*.py`, byte-identical to v1 `syntax_specs/{grep,rm}.py`
`grep`'s three forms; three grammars with one	v1 `doc_sources/man/grep.txt` lines 8–10; the three grammar files, loaded through v1's `caruca.ir.syntax` (2026-10-20)
`grep -e a a` among 122 generated lines	`caruca generate grep --max-count 1`, v1 at `d80324073`, run 2026-10-20
Run 4's probes, files and sources	`naive_llm_caruca/runs/2026-10-20_112956_naive-run-grep-noweb_grep_PROMPT_v1/` (calls 2, 4, 5, 6, 7) and its `analysis/analysis.md`
Run 5's probes and files	`naive_llm_caruca/runs/2026-10-20_121932_naive-run-rm_rm_PROMPT_v1/` (calls 2, 3, 4) and its `analysis/analysis.md`
PaSh's `grep` annotation handles `-e` / `-f`	`naive_llm_caruca/samples/grep/pash/InputOutputInfoGeneratorGrep.py` (from `pash-annotations` 0.2.4)
The agent's setup and prompt	`naive_llm_caruca/README.md`; `ai_docs/tasks/016_naive_claude_code_arm.md`

2. A command beyond the core utilities: `binwalk`

Section 1's commands are GNU core utilities, and in both runs the agent had a finished example for the command in front of it. Run 6 removes both conditions. `binwalk` scans binary files (firmware images, disk images, archives) for embedded files and executable code, and can extract what it finds. It is not one of the 120 commands Caruca has syntax specifications for (`syntax_specs/` holds 120 files), so there is no Caruca reference, model output or annotation for it, and none of the four examples resembles it. It was installed from Ubuntu's package archive, as a user would install it. The setup was otherwise that of runs 4 and 5: the same first-version prompt, model, virtual-machine image and no web access.

Summary

- **Whether a command is opaque depends on how it is packaged.** The installed `binwalk` is a four-line Python script that loads binwalk's Python package. The agent read that package and ran binwalk's own option-registration code to list every option with its type, and wrote its grammar from that list. Caruca's subject is opaque components; many commands installed from a package manager are not.
- **The documentation lags the program.** The manual lists 49 options and describes version 2.3.2; the installed program lists 51 and reports 2.3.3; the package is 2.3.4. As in case study A, a documentation-only syntax step cannot see the two missing options.
- **binwalk's behaviour lives in its input's contents.** It reports only what carries a known format signature. Run on the eight files Caruca fills its test folders with, binwalk found nothing in seven and only the image's own header in the eighth. The agent's first probe built a file worth scanning (a gzip stream followed by a tar archive).
- **Two grammar details matter to Caruca's later stages.** A correct integer range in the agent's grammar would make Caruca's generator try 65,536 values for one option; and binwalk can run arbitrary programs (`-D type:ext:cmd`), which Caruca's own grammars would mark with their `Command` type and the agent's grammar does not.
- **What it suggests:** treat file content as part of the environment model for commands that inspect formats; sample typed ranges instead of enumerating them; type command-valued arguments as `Command` ; and state, per command, whether the evaluation treated it as opaque.

2.1 The run

RUN	COMMAND	MODEL	WEB	TIME · COST (LIST)	USED AS EVIDENCE
6	`binwalk` (Ubuntu package 2.3.4+dfsg1-5)	Claude Fable 5.1, medium effort	removed, and forbidden in the prompt	193 s · $2.12	yes

The agent made 18 tool calls. It read the formats and examples, ran `--help` , read the manual, read binwalk's installed source (calls 6 to 9), ran the binary 30 times in a scratch folder, wrote all four files in one call, then tested them and made one correction after four more runs. It used no network.

2.2 Where the syntax came from

The prompt pointed the agent to the binary and the manual. Its fifth call showed the binary is a Python script (`from binwalk.__main__ import main`), and from then on the installed package was its main source. In call 9 it imported binwalk's module registry and printed each module's command-line option table: every short and long spelling, whether the option takes a value, and the value's Python type (`int` , `float` , `str` , or `list` for an option that may be repeated). Its syntax file and PaSh table follow that table one for one. It compared the table with the manual and with `--help` :

SOURCE	VERSION IT STATES	OPTIONS LISTED
Ubuntu package (`dpkg`)	2.3.4+dfsg1-5	—
`binwalk --help`	v2.3.3	51
Manual page (generated by `help2man` , September 2021)	2.3.2	49: lacks `-0` / `--run-as` and `-1` / `--preserve-symlinks`
binwalk's own option table (call 9)	—	51 (plus one hidden option, which crashes from the shell)

Two observations follow.

- **The source answered the syntax question better than the manual did.** For an interpreted command, the option parser's own table is the definitive list of spellings and value types, and it was on the machine. A documentation-based syntax step, given the bundled manual, would have missed the two options added after 2.3.2.
- **But the agent's check could not fail.** Its main test compared the written files with the same option table they had been written from. It ran 16 of the 51 options on the binary at least once; the 35 never run include the ones with the largest effects: `-D` (runs a command for each match), `-r` (deletes extracted files), `-M` (extracts recursively), `-0` (runs extraction tools as another user) and `-1` (keeps symbolic links that point outside the extraction folder).

2.3 What binwalk needs from the environment

Caruca's configuration generator pairs each invocation with a file-system state and fills files from a fixed content pool. The paper (section 4.3) describes the pool as text, math formulas, JSON and "a set of PNG images ranging from 2kb to 4MB"; the code at `d80324073` ships eight files (`ir/contents.py`, `data/`): two text passages, two arithmetic files, two time files, one JSON file and one JPEG image. The default content level, `simple`, uses the text only.

We ran the installed `binwalk` on each of the eight files, in the virtual machine after the session:

CONTENT FILE(S)	BINWALK RESULTS
`human_1.txt`, `human_2.txt`, `math_1.txt`, `math_2.txt`, `time_1.txt`, `time_2.txt`, `json.json`	none
`usenix.jpg`	1: the JPEG header at offset 0 (the file's own format)

So at the default level every invocation Caruca generated for `binwalk` would print an empty table, and extraction, entropy plotting and file comparison would have nothing to act on. The paper's coverage section states the assumption this meets: path normalization holds because "a command behavior depends not on a specific path name, but rather the identified location type (e.g., file, directory, empty)" (section 7.3). For `binwalk` the location type is the same in every case; what changes the behaviour is what is inside the file.

This cuts both ways. Given an input that triggers extraction, Caruca's tracing would record what the agent's output cannot express: `binwalk -e fw.bin` creates `_fw.bin.extracted/` in the current folder. The agent observed this, but PaSh's input/output interface (`pash-annotations` 0.2.4) has calls only for operands, option arguments and standard input and output, so its annotation records the folder in a comment alone. Traced reads and writes are where Caruca's design is stronger than the agent's.

2.4 Two grammar details with consequences for the pipeline

A typed range. The agent wrote binwalk's status-server port as `Integer(flag="-s", min=0, max=65535)`, which is correct. Caruca's `Integer` generates every value in its range (`ir/syntax.py`, lines 263–269; the default range is –1 to 1, which is what the paper's section 4.2 describes). Fed the agent's grammar, Caruca's own generator (`string_invocations`, at most one option per invocation) produces 65,659 command lines, of which 65,536 are `binwalk -s <port>`, each starting the status server on a different port. Without `-s` it produces 122, the same order as `grep`'s 121 by the same method. None of Caruca's 120 grammars uses `min` or `max`, so the question has not arisen with hand-written grammars; it arises as soon as a model writes a correct range.

An option that runs programs. `binwalk -D type:ext:cmd` runs `cmd` for every match: after the session we ran `binwalk -q -D "gzip:gz:touch RAN_%e" c.bin` and it created `RAN_p.txt.gz`. Caruca's grammar language already has a type for this: `Command` (`ir/syntax.py`, line 387), which its own grammars use for option values such as `sort --compress-`

`program`, `split --filter`, `pandoc --filter` and `install --strip-program`. The agent typed `-D`'s value as free text, and none of its four files says that binwalk can run other programs. The paper names such commands (`xargs`, `nohup`) as the case where its string normalization does not hold (section 7.3).

2.5 What this suggests for Caruca

Each suggestion extends a mechanism Caruca already has.

1. **Make content part of the environment model for format-inspecting commands.** The content pool already varies text, maths, JSON and images. Adding a few format-bearing samples (a compressed stream, an archive, a small file-system image) would let a command like `binwalk` exercise its extraction paths, and Caruca's tracing would then record the files they create. The syntax step could mark a command as content-sensitive, much as it already types arguments, to select them.
2. **Sample typed ranges instead of enumerating them.** Generating the two endpoints and a value inside the range, in the spirit of the −1, 0, 1 the paper uses for integers, keeps `min` / `max` safe for model-written grammars without losing the bound.
3. **Type command-valued arguments as `Command`, and report them.** The type exists and Caruca's references use it. A grammar check that flags option values described as commands in the manual would catch cases like `-D`, and the evaluation could list such commands with `xargs` and `nohup` as outside string normalization.
4. **State the evidence condition per command.** Whether a command was treated as opaque (binary and documentation only) or its source was available changes what a syntax step can know. For interpreted commands the option parser's own table is a third yardstick for syntax, alongside the reference and the binary of section 1.5.

2.6 What this section does not claim

- **One run, one command.** binwalk ran once, with one prompt version. It has no Caruca reference, so nothing here is scored.
- **Some checks are ours, not the agent's.** The package version, `-D`'s command execution and the runs on Caruca's content pool were checked by us in the same virtual machine after the session; the session log does not contain them.
- **The 65,536 figure is Caruca's generator applied to the agent's grammar.** It says nothing about Caruca's own grammars, none of which uses a range.
- **No claim that most commands ship as source.** binwalk is one interpreted command; many packaged commands are compiled.
- **The agent repeated section 1's weaknesses.** It isolated nothing beyond one scratch folder (whose creation was not checked, and one probe wrote a cache file outside it), tested its files mainly against its own source, stated "Stdin is never read" although its own probe read `/dev/stdin` from a redirected file, and did not mention `-D`.

Evidence

CLAIM	SOURCE
binwalk is not among Caruca's grammars	v1 `caruca/src/caruca/syntax_specs/` : 120 grammar files, none for `binwalk`
The binary is a Python script; call 9's option table; 51 options; manual 2.3.2 with 49; `--help` v2.3.3	`naive_llm_caruca/runs/2026-10-20_133813_naive-run-binwalk_binwalk_PROMPT_v1/` (calls 5, 6–9)
Package 2.3.4+dfsg1-5	`dpkg -s binwalk` in the run's virtual machine, 2026-10-20, recorded in the run's `analysis/analysis.md`
30 runs before writing, 4 after; 16 of 51 options run; the circular test	the run's transcript (calls 10, 11, 16, 17) and `analysis/analysis.md` section 2.4
Caruca's content pool	v1 `caruca/src/caruca/ir/contents.py` lines 11–52 and `data/` , commit `d80324073` ; the paper, section 4.3
binwalk on the pool: nothing in seven files, the JPEG header in one	binwalk 2.3.3 in the run's virtual machine, 2026-10-20
`-D` runs a command	`binwalk -q -D "gzip:gz:touch RAN_%e" c.bin` in the run's virtual machine, 2026-10-20
`Integer` enumerates its range; `Command` type	v1 `caruca/src/caruca/ir/syntax.py` lines 263–269 and 387
`Command` used for option values in Caruca's grammars	v1 `syntax_specs/{sort,split,pandoc,install}.py`
Generator counts (65,659; 122	v1's `caruca.cli.generate.string_invocations` at `d80324073` , run on the agent's `output/binwalk.py` from a scratch package, 2026-10-20

CLAIM	SOURCE
without `-s`; `grep` 121)	
PaSh's input/output interface	`pash_annotations/annotation_generation/annotation_generators/InputOutputInfoGenerator_Interface.py`, `pash-annotations` 0.2.4
Path normalization and higher-order commands	the paper, section 7.3